Bachelor of Science in Data Science

Course Code: DS3003 & DS3004

Course Name: Data Warehousing & Business Intelligence

Semester-Fall, 2025

Practical Project on Walmart Data

Building and Analysing a Near-Real-Time Data Warehouse

Total Marks: 100
Weight in grade: 15%

Due date: Monday, 17 Nov 2025, midnight

1. Assessment Task

The student is required to design, implement, and analyze a **near-real-time Data Warehouse** (DW) prototype for **Walmart**. This task involves the creation of a system capable of integrating and processing transactional data efficiently to enable timely business insights and decision making.

2. Project Overview

Walmart, one of the world's largest retail chains, serves millions of customers daily across its stores and online platforms. To optimize sales and enhance customer satisfaction, Walmart needs to analyze shopping behavior in **near real-time**, enabling dynamic promotions and personalized offers. An abstract level overview of Wallmart DW is shown in Figure 1.

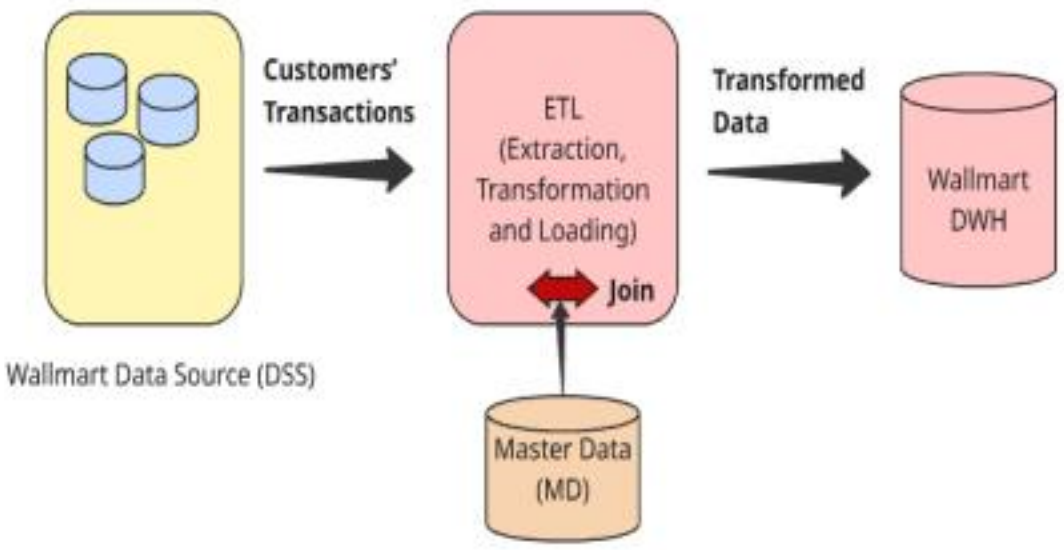


Figure 1: An overview of Wallmart DW

To achieve this near-real-time functionality, the system must include **ETL (Extraction, Transformation, and Loading) tools** capable of processing data streams dynamically. Since data generated by customers is often **incomplete** for DW requirements, it must be **enriched during the transformation phase** of ETL.

For example, additional details such as customer demographics or product attributes can be integrated from Master Data (MD) as illustrated in *Figure 2*.

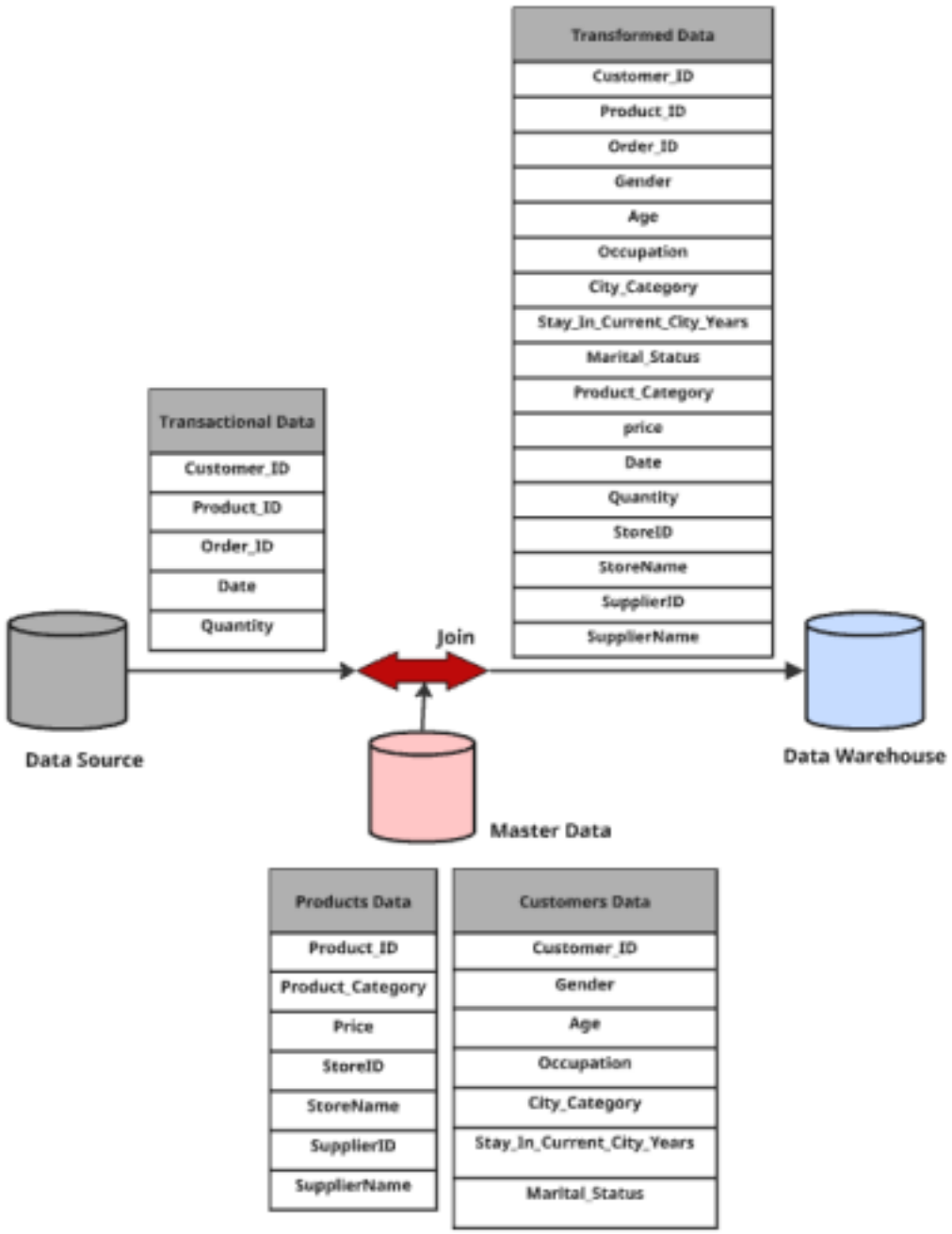


Figure 2: Data Enrichment Process

To implement this **data enrichment** feature, a **Stream-Relation Join Operator** is required in the transformation phase. Several algorithms exist to perform this type of join operation; In this project, you will **implement HYBRIDJOIN (Hybrid Join) algorithm** in **Python**, enhancing its functionality to support near-real-time DW operations efficiently.

3. Data Warehouse Schema

Dimension Tables:

- · **Customer Dimension (customer_dim)**: customer_id, gender, age, occupation, city_category, stay_in_current_city_years, marital_status

- · **Product Dimension (product_dim)**: product_id, product_category, price, store_id, supplier_id, store_name, supplier_name

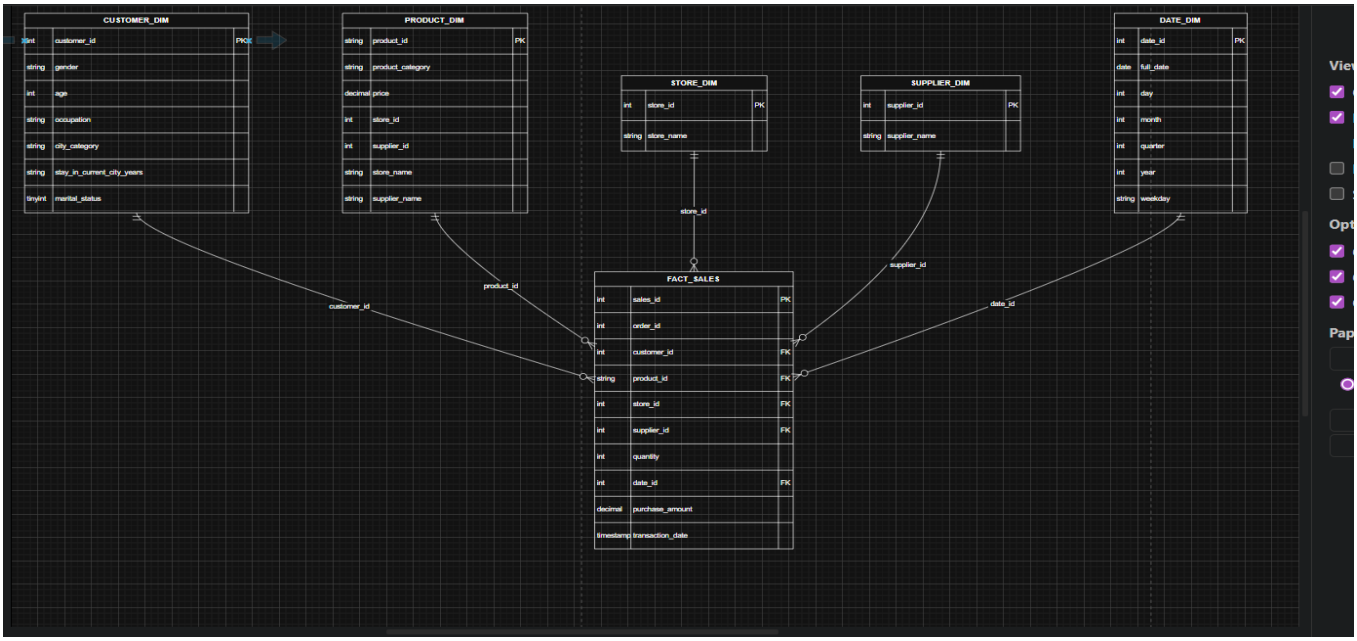
- · **Store Dimension (store_dim)**: store_id, store_name

- · **Supplier Dimension (supplier_dim)**: supplier_id, supplier_name

- · **Date Dimension (date_dim)**: date_id, full_date, day, month, quarter, year, weekday

Fact Table:

· **Fact Sales (fact_sales)**: order_id, customer_id, product_id, store_id, supplier_id, date_id, quantity, purchase_amount, transaction_date.Primary and foreign keys ensure integrity and optimize join operations.



3. HYBRIDJOIN

HYBRIDJOIN is a stream-based join algorithm designed for scenarios like near-real-time data warehousing, where you need to join a fast-arriving, potentially bursty data stream (S) with a large, disk-based relation.

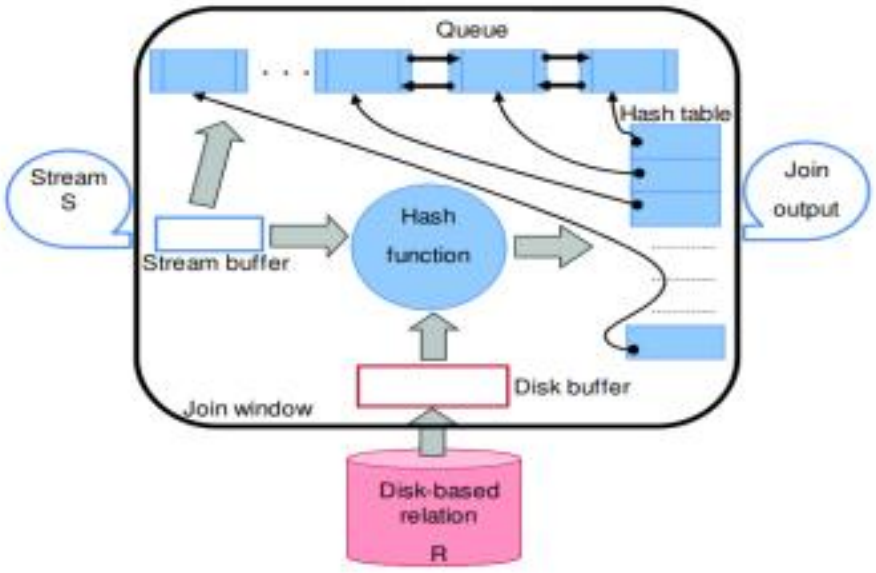


Figure 3: Data Structures used in HYBRID JOIN

Working Principle

HYBRIDJOIN is a stream-based join algorithm designed to efficiently combine a continuous, potentially bursty data stream (S) with a large, disk-based relation (R.

It begins by initializing memory for its key components: a hash table with hS slots, a queue (a doubly-linked list for random deletions), a disk buffer, and a stream buffer. The process runs in an endless outer loop where it first checks the stream buffer for incoming tuples, loading up to w (the number of available hash table slots) tuples into the hash table and adding their join attribute values to the queue. Each tuple is hashed using a function that maps its join key (e.g., Customer_ID) to a slot, allowing multiple matches in a multi-map structure. After loading stream data, the algorithm uses the oldest key from the queue to index into R, loading a relevant partition (size vP) into the disk buffer, leveraging R's indexed and sorted nature to minimize I/O.

In the inner loop, the algorithm probes the hash table with each tuple from the loaded disk partition, generating join outputs for matches, deleting matched stream tuples from the hash table and their corresponding queue nodes, and incrementing w by the number of vacated slots. Unmatched tuples persist in the hash table and queue. The queue shrinks or grows based on join successes or new stream arrivals.

HybridJoin ETL Explanation

The `HybridJoin` Python program extracts transactional data from CSV files, enriches it using master data (customer and product), and loads it into the DW.

Key Features:

1. **Master Data Loading:**
   - Reads customer and product master CSVs.
   - Creates in-memory indexes for fast lookup.
2. **Age Parsing:**
   - Converts age ranges (e.g., '26-35') to average integer.
3. **Hash Queue Mechanism:**
   - Uses a hash table and doubly-linked queue to buffer tuples.
   - Evicts oldest tuples when buffer is full.
4. **Database Handling:**
   - Connects to MySQL DW.
   - Loads dimension tables (customer_dim, product_dim, store_dim, supplier_dim, date_dim).
   - Inserts fact_sales data in batches.
5. **Stream Processing:**
   - Reads transactional data in chunks.
   - Enriches each transaction and commits in batches.
6. **Final Flush:**
   - Processes remaining tuples in hash table and completes ETL.

Workflow Diagram:

1. Stream CSV → 2. Hash Queue → 3. Dimension Enrichment → 4. Fact Table Load

Benefits:

- Efficient ETL with memory buffering.
- Prevents duplicate inserts.
- Batch commit for performance.

5. Shortcomings of HYBRIDJOIN Algorithm

1. **Memory Usage:**
    - The hash table stores multiple tuples in memory; for extremely large datasets, it may require high memory.
2. **No Parallelization for Large Joins:**
    - While it handles batch loading efficiently, it does not fully exploit multi-threading for joins between massive fact and dimension tables.
3. **Dependency on Master Data Quality:**
    - Missing or inconsistent master data can cause incomplete transformations or null inserts in fact tables.

6. Star Schema

```
erDiagram

  CUSTOMER_DIM ||--o{ FACT_SALES : "customer_id"

  PRODUCT_DIM ||--o{ FACT_SALES : "product_id"

  STORE_DIM ||--o{ FACT_SALES : "store_id"
```

```
SUPPLIER_DIM ||--o{ FACT_SALES : "supplier_id"

DATE_DIM ||--o{ FACT_SALES : "date_id"



CUSTOMER_DIM {

    int customer_id PK

    string gender

    int age

    string occupation

    string city_category

    string stay_in_current_city_years

    tinyint marital_status

}



PRODUCT_DIM {

    string product_id PK
```

```
    string product_category

    decimal price

    int store_id

    int supplier_id

    string store_name

    string supplier_name

}



STORE_DIM {

    int store_id PK

    string store_name

}



SUPPLIER_DIM {

    int supplier_id PK
```

```
    string supplier_name
}

DATE_DIM {
    int date_id PK
    date full_date
    int day
    int month
    int quarter
    int year
    string weekday
}

FACT_SALES {
    int sales_id PK
```

int order_id

int customer_id FK

string product_id FK

int store_id FK

int supplier_id FK

int quantity

int date_id FK

decimal purchase_amount

timestamp transaction_date

}

7. Lessons Learned

- Star schema design improves query performance and simplicity.
- Efficient ETL requires batch processing and memory management.
- Dimension tables maintain consistency and support complex analytics.

OLAP queries reveal actionable insights for business strategy

8. Conclusion

This project demonstrates: - Efficient ETL with HybridJoin for large datasets. - A well-structured Data Warehouse using star schema. - OLAP analysis for product, customer, store, and supplier insights.

HybridJoin + OLAP queries provide a complete solution for decision-support analysis in retail data warehouses.